Software Process Management

Lecture 11

Process Modelling Languages

- In this lecture we will be discussing:
- Why Process Modelling Languages?
- What are the notations and tools for PM
- Brief overviews about a PML (MVP-L)



Introduction

- Process Modeling Languages (PMLs) came up in 1980s (Osterweil's "Software Processes are Software too")

- Formalization of the software development process enables:

- **Precise** specification of software development processes
- **Tool-supported** editing and maintenance of process models and generation of process descriptions
- **Automation** of software development processes (e.g., for process engines or process-sensitive software development environments)
- **Standardization** of software process descriptions
- **Common understanding** of process models / process descriptions
- **Instrumentation** of software development processes with measures

Evaluating PM Languages

- **Process Programming vs. Process Improvement**

- A major distinction can be made between process modelling notations for the implementation of processes (i.e., process programming) and notations for the conceptual modelling of processes (i.e., process improvement).
- Process programming notations focus on a representation for interpretation and execution by machines.
- Process improvement notations focus on representation of real-world concepts and provision of a representation that can be interpreted by humans.

- **Hidden vs. Guiding**

- When the process model is used, the representation of the process models can be hidden or presented to the process user.
- When hidden, the process instantiation is completely encoded in the process models or tools that support process enactment; only filtered information is provided.
- If used for guiding, the process models themselves are used to inform the user and to provide guidance during process instantiation.

- **Prescriptive vs. Proscriptive**
- In the early days of software process research, the main focus was placed on automating process execution with the help of software development tools.
- Therefore, the user of such tools would be guided by an execution mechanism in a prescriptive manner. This approach of prescribing the process and thus also the human activities has been subject to criticism and is difficult to implement.
- The proscriptive approach represents a non-restrictive way of formulating processes.
- The process models provide guidance in order to enable performance of the required process steps, but process users have a certain freedom in deciding which actions to take at a particular stage of the project.

- **Single Person vs. Multi-person**
- Software development projects are not performed by a single person and, in consequence, collaboration and cooperation between persons, teams, and organizations is highly relevant.
- Process models should support all these different levels in order to make collaboration and cooperation possible.
- Historically, process representations have evolved from a single-person focus in order to ensure proper application of specific techniques by individuals.
- For the purpose of cooperation, a multi-person focus is needed in order to coordinate the processes of different persons.
- Therefore, a process representation should contain constructs for modelling concepts of collaboration.

Requirements for PM Notations

- A set of requirements have been identified to evaluate Process Modelling Notations.
- The fulfilment of these requirements can be seen as an indicator for the suitability of the notation to support process management for software engineering organizations.
- However, depending on the viewpoint, context and purpose different requirements can have unique importance.

R1- Natural Representation

- Is the mapping of real-world objects on language concepts intuitive and easy for developers?



- Is the schema defined by PML complete?

R2- Support of Quality Management

- AKA *Support of Measurement*
- How are process and products attributes defined?



- How can the process be measured for quality?
- Can quality models be integrated?

R3- Model Adaptability

- A.K.A. *Tailorability of Models*
- Which constructs for defining process variability exist?



- Which constructs supporting process tailoring exist?

- How are process models adapted?

R4 - Formality

- What is the formality level of the language?



- How is the (enactment) semantic defined?

R5 - Comprehensibility

- Is the representation of the model suitable for the actor?



- Is it easy to extract information from the process model?

R6 - Execution & Enactment

- Is the language interpretable?

- Does operation system concepts exist in the language?

Flexibility

- Can process models be changed ‘on the fly’?



- Can states be changed manually?

Traceability

- Is the context of a process easy to analyze?



- Is it easy to navigate through the model?

Framework for PM Notations

- These characteristics and requirements can be used to define a framework that helps to distinguish different process modelling notations and their purpose.
- All elements of this framework are summarized in the following Table.



- For the evaluation of requirements satisfaction, standard approach is to use:
  - (+) represents full,
  - (O) partial, and
  - (-) no
- fulfilment of the respective requirement.

Table 4.1 Characterization framework

Characterization	
Process programming vs. improvement	Prescriptive vs. proscriptive
Hidden vs. guidance	Single person vs. multiperson
Requirements satisfaction	
R1—Natural representation	(+/O/−)
R2—Support of measurement	(+/O/−)
R3—Tailorability of models	(+/O/−)
R4—Formality	(+/O/−)
R5—Understandability	(+/O/−)
R6—Executability	(+/O/−)
R7—Flexibility	(+/O/−)
R8—Traceability	(+/O/−)